Research Project: AVBD Solver Technical Report

MAËL RIOS, Institut Polytechnique de Paris, France

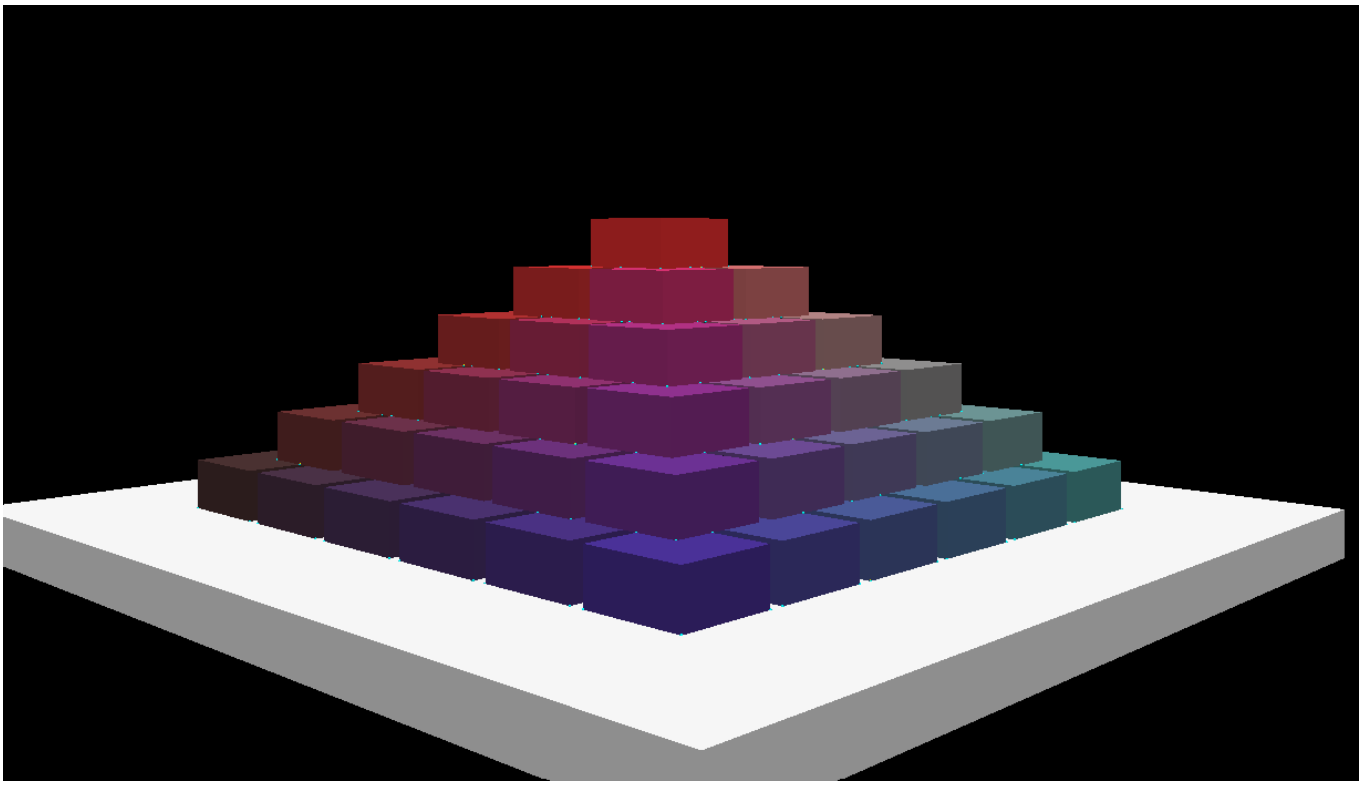

Fig. 1. Pyramid stack, showcasing efficient stacking collision constraint handling by the solver.

ACM Reference Format:
Maël Rios. 2026. Research Project: AVBD Solver Technical Report. 1, 1 (March 2026), 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

In the area of real-time computer graphics applications, specifically interactive ones like computer games, the balance between realism, stability, and speed is crucial.

Modern physics-based solvers need to propose algorithms that can accurately simulate multiple bodies interacting with each other, as stable as possible, and where the balancing can be explicitly modified to favor realism over efficiency, or interactivity over accuracy.

In this report, I will present the main contributions of the paper I chose to build upon, as well as additional implementation details and research thought process that summarizes my work done during this project. First, I will talk about the original AVBD[4] paper and why I decided to implement it. Then, I will share my work done to incorporate Soft Bodies inside the main algorithm and optimization decisions. In the end I present my implementation details and further discussions.

Author's Contact Information: Maël Rios, Institut Polytechnique de Paris, France.

2 What is AVBD

Augmented Vertex Block Descent[4] is a paper that presents a very fast physics-based simulation method, stable under few iterations and that is capable of converging to the implicit Euler solution. It also presents highly parallel potential through GPU compute-shaders and a practical color-graph algorithm, which is ideal if scaling is needed for high end applications.

It shares its base algorithm and primal solver with another paper, Vertex Block Descent[1] which has proven to be really good for soft body simulations. Its greatest strength is to be able to converge quickly with few primal iterations, while its weakness is instability when dealing with high stiffness ratios, the cornerstone of accurate rigidbody collisions or near-rigid soft body simulations and hard constraints.

AVBD builds itself on top of this problem and proposes an augmented Lagrangian formulation. This allows the handling of hard constraints via progressive increases to the desired target stiffness, and also adds a mandatory dual step after each primal step, which makes this solver a primal-dual hybrid method.

2.1 Constraints

In VBD, main interactions are encoded through energy densities. In order to handle constraint-like formulations, such as the ones found in manifold contacts, joints and springs, it is possible to incorporate them through a quadratic energy potential formulation

$$E_j(x) = \frac{1}{2}k_j(C_j(x))^2$$

where k_j and C_j are the stiffness and constraint error for a specific constraint j. The problem is evident: hard constraints require Infinite stiffness, which would immediately lead to convergence issues in this fomulation.

Comes in the Augmented Lagrangian formulation through $k_j = \infty$ as

$$E_j^{(n)}(x) = \frac{1}{2}k_j^{(n)}(C_j(x))^2 + \lambda_j^{(n)}C_j(x)$$

where they incorporate finite stiffness $k_j^{(n)}$ and the dual variable $\lambda_j^{(n)}$ at iteration n. Without going too much into the details, this formulation allows the growth of the finite stiffness up to the target stiffness k_j per iteration while the dual term $\lambda_j^{(n)}$ allows for the constraint to be mostly respected even though the finite stiffness is "small" with regards to the true target.

The finite stiffness and the dual term are then updated at each iteration to steer the solution towards convergence in the following ways:

$$\lambda_j^{(n+1)} = k_j^{(n)}C_j(x) + \lambda_j^{(n)}$$
$$k_j^{(n+1)} = k_j^{(n)} + \beta|C_j(x)|$$

The dual parameter is scaled by the current iteration's finite stiffness, which is in turn increased through a hyperparameter β. This is AVBD's main tunable hyperparameter, which only dictates how fast the constraint converges to the solution. A high value can inject sudden changes in the simulation and therefore extra energy leading to explosive results, whilst a low value never respects the target constraint leading to no visible interactions whatsoever.

2.2 Some Optimizations

AVBD comes with several optimizations, explained one by one in the paper, that allow the algorithm to more accurately converge towards the solution while fixing potential problems due to the addition of this Lagrangian dual term.

Among them are efficient minimal and maximum $\lambda_j^{(n)}$ value clamping, correct and efficient friction computations, potential inter-frame explosive error correction through a second hyperparameter α and adaptive warmstarting to capture the last recorded $\lambda_j^{(n)}$ and $k_j^{(n)}$ for existing constraints in order to obtain faster convergence in the following steps.

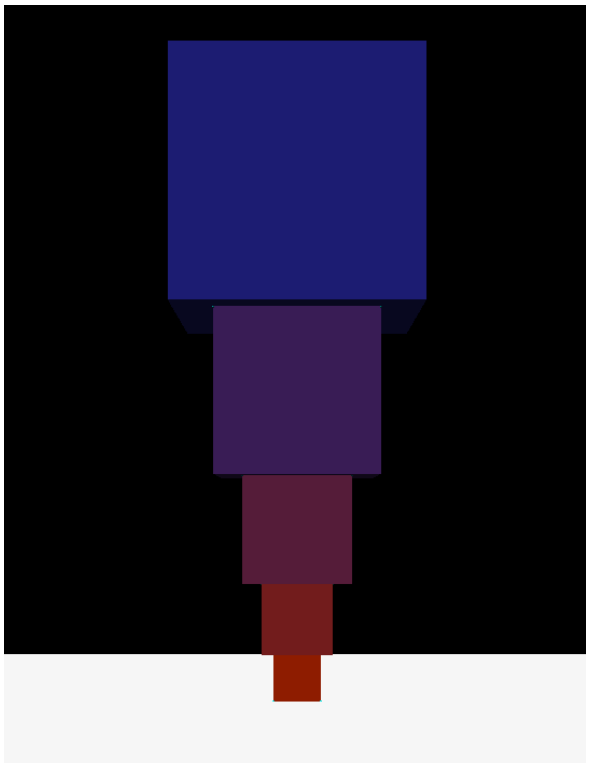

Fig. 2. Rigid Body Stacking under high Mass Ratios, demonstrating VBD stable capabilities.

One optimization that is particularly important is Hessian approximation. VBD eliminates the need for this by skipping solver steps for a particular energy where the hessian is not invertible (not positive definite) and expecting it to become invertible next step. However, when introducing a lagrangian dual variable such as $\lambda_j^{(n)}$ in the quadratic energy formulation, the hessian becomes

$$H_{ij} = k_j^{(n)}\left(\frac{\partial C_j(\mathbf{x})}{\partial \mathbf{x}_i}\right)^T \frac{\partial C_j(\mathbf{x})}{\partial \mathbf{x}_i} + \mathbf{G}_{ij}$$

where the second term $\mathbf{G}_{ij} = \lambda_j^+ \partial^2 C_j(x)/\partial^2 x_i^2$ is the second derivative of the constraint scaled by λ_j^+ and can be indefinite and non-symmetric. They solve this by replacing it by the diagonal matrix

$$\tilde{\mathbf{G}}_{ij} = diag(g_{ij})$$

where each element of the vector g_{ij} is the norm of the vector $G_{ij,c}$ forming a column of G_{ij}. This way the Hessian is symmetric positive definite by construction, allowing a proper use of LDL^T decomposition to solve the system instead of direct inverse. This hessian approximation transforms the solve step into a Quasi-Newton step.

collision detection using x
update colorization
$y \leftarrow x + hv + \Delta t^2 a_{ext}$
$x \leftarrow$ initial guess with adaptive initialization
$\lambda \leftarrow \alpha\gamma\lambda$
$k \leftarrow \max(k_{\text{start}}, \gamma k)$
for $n_{\max}$ iterations **do**
  for each color c **do**
    for each vertex i in color c (in parallel) **do**
      $f_i \leftarrow -\frac{M_i}{\Delta t^2}(x_i - y_i)$
      $H_i \leftarrow \frac{M_i}{\Delta t^2}$
      for each force j affecting vertex i **do**
        if j is hard constraint **then**
          $f_i \leftarrow f_i - \text{clamp}\left(k_j C_j(x) + \lambda_j, \lambda_j^{\min}, \lambda_j^{\max}\right) \frac{\partial C_j}{\partial x_i}$
        else
          $f_i \leftarrow f_i - k_j C_j(x) \frac{\partial C_j}{\partial x_i}$
        end if
        $H_i \leftarrow H_i + k_j \left(\frac{\partial C_j}{\partial x_i}\right)^T \frac{\partial C_j}{\partial x_i} + G'_{ij}$
      end for
      $x_i^{new} \leftarrow x_i + H_i^{-1} f_i$
    end for
    for each vertex i in color c (in parallel) **do**
      $x_i \leftarrow x_i^{new}$
    end for
  end for
  for each force j (in parallel) **do**
    if j is hard constraint **then**
      $\lambda_j \leftarrow \text{clamp}\left(k_j C_j(x) + \lambda_j, \lambda_j^{\min}, \lambda_j^{\max}\right)$
      if $\lambda_j^{\min} < \lambda_j < \lambda_j^{\max}$ **then**
        $k_j \leftarrow k_j + \beta |C_j(x)|$
      end if
    else
      $k_j \leftarrow \min\left(k_j^*, k_j + \beta |C_j(x)|\right)$
    end if
  end for
end for
$v \leftarrow (x - x^t)/\Delta t$

Algorithm 1: *AVBD simulation for one time-step*

2.3 Algorithm details

The entire algorithm can be found described in Algorithm 1, taken directly from [4], and where the red parts are the main elements changed from the VBD formulation.

Among the main changes are the added dual loop (only for hard constraints), updating the dual parameter, and the hessian approximation before the solving of the quasi-newton step. We can also see parts of the other optimizations proposed such as the clamping on the dual during the primal loop and proper warmstarting at the beginning of the step through the third hyperparameter γ, defining how much of the previous step results to incorporate into the next step (ideally 0.95 as we still wish the step to converge towards the solution and not be directly AT the solution in case it was attained before).

This algorithm is incredibly powerful as it allows any force that can be written as a constraint to be added into the solver, as long as we know how to compute its gradient and Hessian (or at least an efficient SPD approximation of it). There is however no mention on how soft-bodies should be implemented in the solver, even though it has the same building blocks as those found in the VBD formulation. Their incorporation became one of the main research directions through the rest of my project.

3 Adding soft bodies

Since the original VBD paper focused specifically on soft body energy densities[1] for its primal solver, it made sense to research how one could incorporate them back into AVBD to obtain a solver capable of handling both constraint-based forces used in rigidbodies and energies for the deformable geometry. Here are the details on how I managed to implement them into the solver, and some researched optimizations for improved stability.

Adding energy densities, such as the ones used by hyper-elastic material models (Neo-Hookean and StVK models in this case), means that I needed to derive their Jacobian and Hessian without being able to pass through the constraint-based formulations proposed by AVBD. This means implementing a totally separate, and still equivalent, object definition in the codebase from constraint forces.

3.1 Discretization

In my solver, soft bodies are discretized with particles as vertices: volumetric bodies use tetrahedral elements (Neo-Hookean), and thin shells use triangular elements (StVK). Each particle is a solver body with only three translational DoF (no rotation possible whatsoever). For a tetrahedral element with vertices $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$ in world space, the deformation gradient is computed relative to the rest configuration $\mathbf{D}_m \in \mathbb{R}^{3\times3}$ precomputed at construction time as well as its inverse:

$$\mathbf{D}_m = \left[\mathbf{p}_1^0 - \mathbf{p}_0^0 \mid \mathbf{p}_2^0 - \mathbf{p}_0^0 \mid \mathbf{p}_3^0 - \mathbf{p}_0^0\right]$$

At each step, the world-space edge matrix $\mathbf{D}_s$ is formed from current vertex positions, and the deformation gradient is computed as:

$$\mathbf{F} = \mathbf{D}_s \, \mathbf{D}_m^{-1}, \qquad J = \det(\mathbf{F}).$$

These two terms will then be used to compute the first Piola-Kirchhoff stress (giving us the gradient of the energy density) and the projected hessian (whose computation I will describe later). Multiplied by the rest volume (rest area for the StVK hyper-elastic model) A_0, this gives us the correct derivatives of the total elastic energy stored in each element.

In addition, from the deformation gradient, the value of the volumetric strain and the Frobenius norm are also derived and stored for an optimization I will describe later. They are computed as:

$$volStrain = |J - 1|, frobNorm = ||F - I||$$

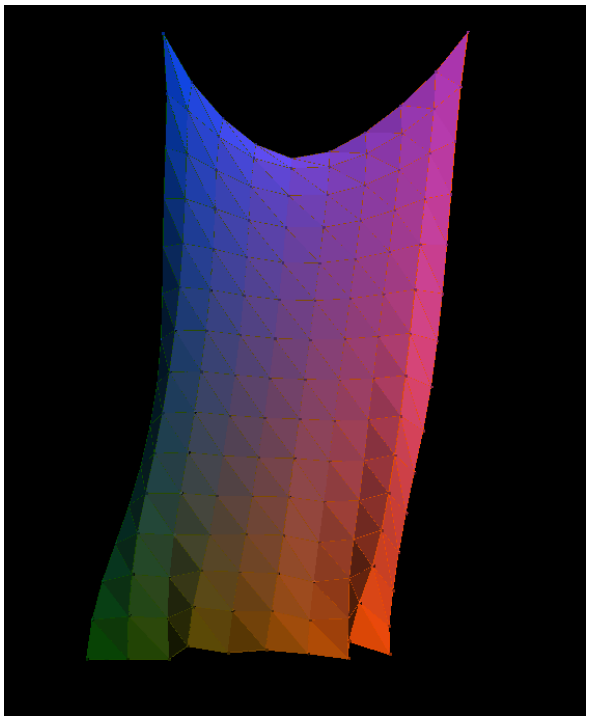

Fig. 3. Cloth simulation using StVK hyper-elastic model.

for the Neo-Hookean model and

$$volStrain = trace(L), frobNorm = ||L||$$

for the StVK model, L defined as the Green-Lagrange strain tensor.

3.2 Stiffness and Incompressibility parameters

The parameters that define how stiff and how incompressible a material is are respectively the Young's modulus E and the Poisson's ratio ν. The former defines the material's resistance to stretching (i.e. its stiffness) while the latter measures its incompressibility, (how much it contracts in the perpendicular direction of a stretching).

These parameters are very useful when describing how a hyper-elastic material reacts to its environment, but they are not used in the computations directly. Instead, derive the lamé parameters from their value with the following equations:

$$\mu = \frac{E}{2(1+\nu)}, \qquad \lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}.$$

3.3 Neo-Hookean Energy Model Specifics

The Neo-Hookean model is used for volumetric tetrahedral elements. The energy density I use is the logarithmic (also called stable) Neo-Hookean formulation:

$$\Psi_{\text{NH}}(\mathbf{F}) = \frac{\mu}{2}(I_1 - 3) - \mu \ln J + \frac{\lambda}{2}(\ln J)^2,$$

where $I_1 = \|\mathbf{F}\|_F^2 = \text{tr}(\mathbf{F}^\top \mathbf{F})$ is the first invariant. I prefer this formulation over its approximation (as described in VBD[1]) because the latter would inflate the elements at low poisson ratios when projecting the Hessian, which should not happen in hyper-elastic materials.

The first Piola-Kirchhoff stress can therefore be computed as:

$$\mathbf{P} = \frac{\partial \Psi}{\partial \mathbf{F}} = \mu \mathbf{F} + \frac{\lambda \ln J - \mu}{J} \text{cof}(\mathbf{F}),$$

where $\text{cof}(\mathbf{F}) = J\mathbf{F}^{-\top}$ is the cofactor matrix, computed explicitly to avoid numerical issues when J is small.

3.4 Saint Venant-Kirchhoff Energy Model Specifics

The StVK model is used for triangular shell elements. It is defined through the Green-Lagrange strain tensor:

$$\mathbf{E} = \frac{1}{2}(\mathbf{F}^\top \mathbf{F} - \mathbf{I}),$$

where $\mathbf{F} \in \mathbb{R}^{3\times 2}$ is the 3D-to-2D deformation gradient. The energy density is:

$$\Psi_{\text{StVK}}(\mathbf{F}) = \mu \|\mathbf{E}\|_F^2 + \frac{\lambda}{2}(\text{tr}\,\mathbf{E})^2.$$

The first Piola-Kirchhoff stress is:

$$\mathbf{P} = \mathbf{F}(2\mu \mathbf{E} + \lambda\, \text{tr}(\mathbf{E})\, \mathbf{I}_2)\,.$$

StVK is simpler and less expensive than Neo-Hookean but is not appropriate for volumetric bodies due to its low resistance to volume inversions. I use it in clothing simulations, for example.

3.5 Hessian via Analytic Eigensystems

The Hessian at vertex i is the 3×3 second derivative of the energy with respect to the vertex $\mathbf{x}_i$. Computing it by directly differentiating $\mathbf{P}$ (the first Piola-Kirchhoff Stress) would produce a 9×9 F-space Hessian, which would then need to be mapped to vertex space in order to add it to the left-hand side of the LDL^T solve. As stated in [1], there is no guarantee this creates a positive definite Hessian that ends up being invertible.
Instead, I use the analytic eigensystem decomposition described in Smith, de Goes and Kim (2019)[7], which computes the Hessian already decomposed into its eigenvectors and eigenvalues in F-space, which enables direct SPD projection before reconstruction.

3.5.1 F-Space Eigenmodes. For isotropic energies, the 9×9 Hessian in deformation-gradient space can be split into 9 modes with a known structure (4 in 2D). In 3D, these modes fall into three groups: 3 scaling modes, 3 twist modes, and 3 flip modes. An eigenmode (or eigenvector) lets us decompose the hessian into simple components carrying a deformation meaning, so that I can identify which directions are stable and which need special attention to transform an indefinite Hessian into its clean SPD projection.

The scaling modes describe stretching along the singular directions of $\mathbf{F}$. Their values are obtained from a small symmetric 3×3 matrix built from the singular values. The twist modes describe antisymmetric changes between pairs of singular directions. There is one twist mode for each pair of directions. The flip modes describe symmetric changes between pairs of singular directions. Again, there is one mode for each pair.

For StVK triangles, the same idea gives 4 modes instead of 9: 2 scaling modes, 1 twist mode, and 1 flip mode. In that case, the scaling part reduces to a 2×2 eigenproblem.

3.5.2 Vertex Hessian Reconstruction. After computing the modes in F-space, we map each one to vertex space using the shape function gradient. This gives a direction vector for each mode at each vertex (necessary to use the Hessian in the solver LDL^T decomposition solve).

The vertex Hessian is then built by summing the outer products of these vectors, weighted by the corresponding projected eigenvalues. In other words, each mode contributes a rank-1 term, and the full vertex Hessian is the sum of all of them.

This produces a 3×3 Hessian for each vertex, which is then inserted into the upper-left block of the 6×6 final Hessian.

3.5.3 SPD Projection and Eigenvalue Filtering. As said before, the F-space Hessian is not always positive semi-definite. Under strong compression or inversion, some eigenvalues, especially the flip modes and sometimes the scaling modes, can become negative. When this happens, the Newton step may move in a bad direction and make the solve unstable.

To avoid this, it is possible to modify the eigenvalues depending on their projected values before rebuilding the Hessian. This is the desired guarantee to make the reconstructed Hessian positive semi-definite. [2] and [3] propose different ways to transform these eigenvalues, with a stability and convergence performance boost for each reconstructed Hessian.

They define three filtering choices. The first is **clamping**, which replaces negative eigenvalues with a small non-negative value. The second is **absolute**, which reflects negative eigenvalues to positive values and usually gives stronger resistance to inversion (this is the transformation that has given me the best results). The third is an **adaptive** mode, which switches between clamp and absolute based on a trust-region value ρ. This value is computed from local energy decrease per element based on the system total energy, but in practice this complicates the solver more than it helps, as the cases where clamping is prefered to absolute values are in general too niche in the solver for it to make a difference.

Because the filtering is applied directly to the small set of scalar eigenvalues before reconstruction, the final projected Hessian is symmetric positive semi-definite by construction, and I can safely inject it every step into the solver.

In the end, when comparing this with computing the exact hessian and skipping the iterations where the Hessian is not invertible, the latter shows better results in term of performance and stability. This step is realistically more valuable when the material's geometric context is in a rough patch, such as very large stretching and inversion, and it will probably not fix itself on the next iterations.

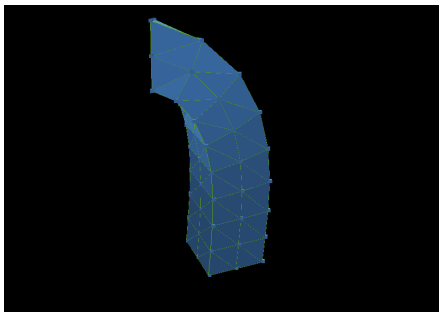

Figure 1

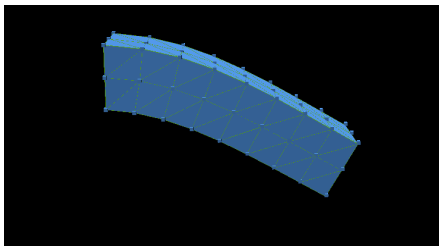

Figure 2

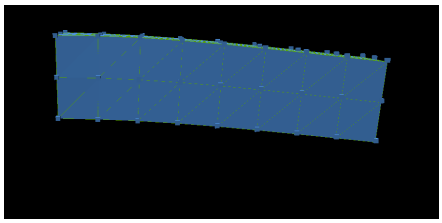

Figure 3

Fig. 4. Comparison of three soft-body beam simulation with increasing Young's modulus and Poisson Ratio. The beams are rooted in world space with hard joint constraints.

But if we manage to handle the inversion well with a proper response, we can take one of these two inconveniences out of the picture.

3.6 Inverted Element Handling

When a tetrahedron becomes inverted, the standard Neo-Hookean energy is no longer valid. To keep the simulation stable, I apply in these degenerate cases a simple penalty-based fallback instead of using the usual gradient and Hessian. This fallback applies a restoring force that pushes the element back toward a valid positive volume. For the Hessian, I use a simple isotropic positive definite matrix, which is cheap to compute and remains stable even in these extreme cases. For StVK triangles, I use a similar idea when the triangle becomes nearly degenerate (below 1% of its rest area). In that case, a simple linear penalty and the same isotropic Hessian does the trick.

4 My implementation details

Many complication arise when trying to implement a research paper from scratch, and even more so when not everything is explicitely

detailed in these papers. I focused most of my attention towards the solver itself, but many orthogonal design choices and implementations had to be made in order to run a simulation to begin with.

Throughout my semester, I started implementing this paper in 2D with Typescript and a WebGPU rendering backend. It took some time to nail-down but it ultimately finished working for most constraints and even for the soft bodies integration. However, I then challenged myself to port the code and create a 3D C++ solver since it would provide more satisfactory results and it could become a better demonstration of AVBD capabilities overall. I will therefore prioritize talking about the 3D implementation details throughout the rest of the paper, since they are almost a 1-1 analogy with my 2D application with extra difficulties tied to the additional degrees of freedom. You can find the original 2D solver **here!**.

4.1 Broad Phase

The paper's reference implementation uses a Linear Bounding Volume Hierarchy (LBVH) for broad-phase collision detection. I did not want to spend too much time implementing it for two reasons: a simple broad-phase collision detection is sufficient to start implementing the paper (even though it will obviously affect scalability) and I had no experience coding such thing in compute shaders. I recently completed a project to compute fast LBVHs with maximized parallel algorithms, so this is a potential addition I could do in the near future. Instead, a simpler insert sort sweep over each body AABB, on the x axis, is used. This way, I can cache the last position orders for each body in a sorted array, and modify it only when their world AABB bounds drift enough for it to make a difference. We then sweep left to right in the array to test pair of intersecting bodies, and a narrow-phase is executed only if the respective AABBs bounds of the mesh intersect each other (they are slightly inflated to discard false negatives).
I guard the creation of manifolds with `isConstrainedTo`(A, B), which walks a cached pair index if the two bodies were already constrained with another force (such as springs or joints).

4.2 Narrow Phase Collision Detection

Rigid bodies are not all treated with the same narrow-phase routine. Instead, the collision method is selected from the model types of the two bodies. If both bodies are spheres, a dedicated sphere-sphere test is used and a single contact point is sufficient (and very efficient). For example, if one body is a sphere and the other a box, a dedicated sphere-box routine is used, again producing a single manifold point. If both bodies are boxes, I keep a specialized box-box implementation based on oriented bounding boxes (OBBs), since it is cheaper and currently more robust than the fully generic method. In cases of more generic shapes, collision detection falls back to a generic convex hull versus convex hull routine.

For this case, a convex hull is precomputed once at startup for each model type from its mesh geometry and then reused by all instances of that model. The hull is stored in local model space and is transformed at runtime by the instance translation, rotation, and scale. Hull-hull narrow phase uses the generalized Separating Axis Theorem: candidate axes are taken from face normals of hull A, face normals of hull B, and all valid cross products between edges of both hulls. As in the box-box case, the axis with minimum penetration is selected as the contact normal.

	Sphere	Capsule	Hull	Mesh
Sphere	YES	YES	YES	YES
Capsule		YES	YES	YES
Hull			YES	YES
Mesh				NO

Fig. 5. Image taken from a slide in [5]. We understand all different possible dispatches, and how generic Mesh - Mesh collision is not recommended at all for real time applications.

In order to bias the solver towards face contacts, which usually generate richer and more stable contact manifolds, a face-contact bias is applied when testing edge-edge axes: an edge-edge axis is only chosen as candidate if its penetration satisfies

$$d_{\text{edge}} < \rho_f \, d^*_{\text{face}} + \epsilon_f,$$

where d^*_{face} is the best face-contact penetration found so far, $\rho_f = 0.95$, and $\epsilon_f = 0.005\,\text{m}$.

Once the contact type is known, the contact points are generated differently depending on the selected routine. Sphere-based cases use a single witness point manifold. Box and Hull edge contacts are computed from the closest points between the witness edges, while face contacts are obtained by clipping the incident polygon against the side planes of the reference face using the Sutherland-Hodgman algorithm.

The arguments in favor of using the SAT algorithm for one-shot manifold and not GJK for per-frame manifold construction were stability and ease of implementation. AVBD does rely a lot on its manifold contacts found quickly and on stable ground in order for the constraint response and stacking to efficiently apply. Plus, this is largely sufficient in cases where the convex hull of the mesh is not overly complex. I inspired myself from Dirk Gregorius talk [5], whose own engine implements this case-by-case method dispatching. They further explain that triangle-mesh collisions can only be done efficiently if one of the two meshes is static, otherwise they keep it on convex hull grounds.

4.3 Contact Identification and Warm-Starting

Each contact point carries a 32-bit identifier encoding the axis type (2 bits), the index of the separating axis on body A (3 bits), the index on body B (3 bits), and a vertex index (4 bits). At the start of each time step, newly detected contacts are matched against those from the previous frame via this identifier. This way it is possible to keep the previously stored k penalty and dual variable λ, providing the warm-starting of the AVBD augmented Lagrangian proposed optimization.

For new contacts (no matching ID found), the initial penalty is

Fig. 6. Example of rigidbodies with same initial height and velocities, with increasing friction coefficients.

seeded proportionally to penetration depth:

$$k_{\text{seed}} = \text{clamp}\big(\delta \cdot c_k,\ k_{\text{min, }10^5}\big),$$

where c_k is a user-tunable scene parameter (`onPenetrationPenalty`).

4.4 Friction

Each body carries a scalar friction coefficient μ_k. The effective friction for a contact pair is the geometric mean $\mu = \sqrt{\mu_A \mu_B}$.

The AVBD paper describes friction using the same augmented Lagrangian framework as contacts, and I implemented Coulomb friction cone bounds on the dual variables of the tangent and bitangent directions (as described earlier):

$$\big|(\lambda_{t_1}, \lambda_{t_2})\big| \le \mu\,|\lambda_n|.$$

Static friction is detected if the tangential magnitude is inside the Coulomb cone, and the tangential initial constraint $\|C_{0,\tan}\| < \varepsilon_s$ where $\varepsilon_s = 0.01$ m. That is the principal criterion to define contacts as sticking, which means the relative contact positions on the two bodies r_A, r_B are frozen to nullify movements that do not break the Coulomb cone.

4.5 Primal System and LDL$^\top$ Solve

Each body's primal system is a 6×6 symmetric positive semi-definite linear system (3x3 in 2D, you need to add a third axis and two extra rotational unknowns to get the 3D system) assembled as

$$\mathbf{H}_i = \frac{\mathbf{M}_i}{h^2} + \sum_c k_c \mathbf{J}_c \mathbf{J}_c^\top + \tilde{\mathbf{G}}_c, \qquad \mathbf{f}_i = \frac{\mathbf{M}_i}{h^2}(\mathbf{x}_i - \mathbf{y}_i) + \sum_c \mathbf{J}_c f_c,$$

where $\mathbf{M}_i = \text{diag}(m\mathbf{I}_3,\ \mathbf{R}\mathbf{I}_{\text{body}}\mathbf{R}^\top)$ is the block-diagonal generalized mass matrix (cached at step start), $\mathbf{J}_c \in \mathbb{R}^6$ is the generalized Jacobian. You can find here again, specific for hard constraints, the quasi-Newton Hessian correction $\tilde{\mathbf{G}}_c = \text{diag}(\|\mathbf{H}_c^{(j)}\| \cdot |f_c|)$. The system is solved using Eigen's Cholesky LDL$^\top$ decomposition, with a single solver object pre-allocated per `Solver` instance to avoid repeated heap allocation in the inner loop.

4.6 Stiffness Ramping Rate

The AVBD paper uses $\beta = 10$, but in practice I find way better results with $\beta = 10^5$, which is a significantly more aggressive ramping rate. This does improve the convergence rate mostly in rigidbody manifold contacts and joint hard constraints, but I am not sure why my solver works with such a different ramp up value...

4.7 Fracture

Each constraint row carries a fracture threshold τ_f on the magnitude of λ. If $|\lambda_c| \ge \tau_f$ during the dual update, the force element should be permanently disabled, effectively breaking the constraint. However, I found that this often leads to the explosion of neighbor constraints, specifically in long joint/spring chains. I suppose this is due to how these constraints ramp up their stiffness up to the target in order to minimize the constraint violation, which in the case of fracture, could create a big stiffness propagation discontinuity. This is still something I am investigating and have not been able to accurately fix.

A possible contribution (in the case that this is an AVBD weakness) could potentially be the introduction of a propagating breaking factor that would ease the stiffness and dual variables of neighbor constraints when one chain part breaks. This way, a fracture would not create such an immediate discontinuity and the other constraints would have the time to reset their values and converge towards the new constraint violations introduced by the break.

4.8 Potential future implementations

When scaling the sizes of a scene and the number of bodies at play, I found out by profiling that most of the computation heavy code resides in the primal solve part. It roughly takes 90% of the total step time. This is evidently the biggest possible bottleneck CPU side, and is probably why the paper focuses on making it as parallel as possible. Therefore the most straightforward optimization becoming a big performance win would be the parallel broad-phase collision detection into proper graph coloring and parallel compute primal and dual loop solves. Since VBD and AVBD are Gauss-Seidel type iterative methods, graph coloring is used to detect which body groups can be solved in parallel without impacting convergence. Compared to PBD, VBD and AVBD color vertices (bodies) instead of forces/constraints as described in [1] making the number of groups smaller and improving efficiency.

Once this is added, implementing more complex rigidbodies and narrow-phase collision detection would create more visually-appealing scenes and could better demonstrate AVBD capabilities for mantaining complex stacking manifold constraints. In order to make this work, SAT box collision detection wouldn't make the cut anymore, and I would need to implement a more precise triangle-triangle detection system, which is probably more efficient in parallel. However, once such a system is implemented, soft-body - rigidBody collision detection would be more realistic, since we rely (for now) on the rigidbodies interacting with the small vertices of the soft bodies to create collisions.

Finally, even though the convergence time and stability of soft bodies is greatly improved with eigen-projection methods on their Hessian, I would like to further investigate how AVBD formulations can further help with their proposed Lagrangian Augmentation.

Indeed, the formulation of energy densities in hyper-elastic material models differ greatly from its constraint-based counterpart, which is optimized through a Lagrangian Augmentation. It is therefore really hard to incorporate AVBDs principal contributions to soft-bodies, which is really confusing since they suffer from the same high stiffness ratio problems that demanded special treatment

for hard constraints to begin with.

Because of this, we could imagine further contributions expanding the use of special dual terms for energies with high Young's modulus and Poisson Ratios. This way, their incorporation into the solver would seamlessly capitalize on the main benefits AVBD presents, all the while making a Lagrangian Augmentation solver a practical design choice.

In order to try and implement this, I decided to store the strain $s_j^{(n)}$ computed from each energy's deformation matrix at iteration n (talked about in section 3) and use it to ramp-up a new dual variable $\tilde{\lambda}_j^{(n)}$. This strain value would mirror the stiffness per iteration in the constraint-based system, and would scale the Hessian's contribution up to the target stiffness. The dual $\tilde{\lambda}_j^{(n)}$ would then be used to inject into each projected Hessian a diagonal term to stabilize the LDL^T solve and attenuate energy injection into the system (specifically for StVK based densities that quickly tend to explode with high stiffness ratios). The ramping of this dual variable takes advantage of AVBD's primal-dual formulation and is done inside the already existing dual loop. However, at the time of testing this technique, I had not implemented eigen-projection methods on the hessians, which meant that the improvement on the system's stability could not be accurately tested. But I do believe there is potential in applying AVBD formulations to isotropic energies in order to adress these high stiffness ratio problems.

5 Personal Conclusions

I initially brought the AVBD paper into the Research Project discussion because I wanted to implement a solver in order to create fun and satisfying looking renders. After reading through the main paper, I had absolutely no idea how hard making a first rough demo would be. I hit a lot of frustrating walls trying to figure out proper collision detection, solver instability profiling and implementing the paper as intended. Turns out there are a lot of prerequisites that are not explicitly laid out in these papers because the work they are built upon is big in volume as is, and I even learned a lot trying to re-implement this in 3D with additional degrees of freedom.

Most importantly, I really enjoyed the chain of thought research brings to the table. This project started out by trying to incorporate AI inference for quicker convergence into the solver as an additional contribution, and I ended up spending hours reading and learning hyper-elastic model formulations and optimizations for proper soft-body simulations [6].

I have however still not been able to solve all bugs and inefficiencies in my solver. I still get some (rare) body interpenetration, high stiffness soft bodies are still injecting some energy into the solver and hard joint constraint chains still explode rather easily. It is also hard to find whether some of these aspects are weaknesses of the solver itself or implementation errors: for example, I still do not know for sure whether the explosions after fracture are due to an error on my side or if they are a real weakness we could turn into a contribution. Nevertheless, I believe my knowledge in building quasi-newton solvers and Gauss-Seidel iterations has increased tremendously, and I hope I will be able to apply this knowledge for future research directions or industry related work.

References

[1] Anka He Chen, Ziheng Liu, Yin Yang, and Cem Yuksel. 2024. Vertex Block Descent. arXiv:2403.06321 [cs.GR] https://arxiv.org/abs/2403.06321

[2] Honglin Chen, Hsueh-Ti Derek Liu, Alec Jacobson, David I.W. Levin, and Changxi Zheng. 2024. Trust-Region Eigenvalue Filtering for Projected Newton. In *SIGGRAPH Asia 2024 Conference Papers* (Tokyo, Japan) *(SA '24)*. Association for Computing Machinery, New York, NY, USA, Article 120, 10 pages. doi:10.1145/3680528.3687650

[3] Honglin Chen, Hsueh-Ti Derek Liu, David I.W. Levin, Changxi Zheng, and Alec Jacobson. 2024. Stabler Neo-Hookean Simulation: Absolute Eigenvalue Filtering for Projected Newton. In *ACM SIGGRAPH 2024 Conference Papers* (Denver, CO, USA) *(SIGGRAPH '24)*. Association for Computing Machinery, New York, NY, USA, Article 90, 10 pages. doi:10.1145/3641519.3657433

[4] Chris Giles, Elie Diaz, and Cem Yuksel. 2025. Augmented Vertex Block Descent. *ACM Transactions on Graphics (Proceedings of SIGGRAPH 2025)* 44, 4 (08 2025), 12 pages. doi:10.1145/3731195

[5] Dirk Gregorius. 2015. Robust Contact Creation for Physics Simulations. Game Developers Conference (GDC) slides. https://media.steampowered.com/apps/valve/2015/DirkGregorius_Contacts.pdf Presentation slides.

[6] Theodore Kim and David Eberle. 2022. Dynamic deformables: implementation and production practicalities (now with code!). In *ACM SIGGRAPH 2022 Courses* (Vancouver, British Columbia, Canada) *(SIGGRAPH '22)*. Association for Computing Machinery, New York, NY, USA, Article 7, 259 pages. doi:10.1145/3532720.3535628

[7] Breannan Smith, Fernando de Goes, and Theodore Kim. 2019. Analytic Eigensystems for Isotropic Distortion Energies. *ACM Trans. Graph.* 38, 1 (2019), 3:1–3:15. doi:10.1145/3241041